

AI POWERED THREAT DETECTION SYSTEM

Real-World Production Deployment & Infrastructure Guide

1. Enterprise Production Architecture

Deploying the **AI Powered Threat Detection System** into live enterprise networks requires dual-mode operation: (1) Cloud Web Console Tier deployed on Vercel Serverless / AWS for global SOC monitoring, and (2) On-Premise Packet Capture Taps deployed on network switches / gateways using physical network socket interfaces.

2. Option A: Vercel Serverless Cloud Deployment

- Step 1: Code Repository Preparation**
Ensure `api/index.py`, `vercel.json`, and `package.json` are present in project root directory.
- Step 2: Connect GitHub to Vercel**
Log into Vercel Dashboard, click **Import Project**, and select the GitHub repository `ai-network-attack-detection`.
- Step 3: Serverless Runtime Configuration**
Vercel automatically detects `@vercel/python` runtime and builds WSGI handler in `api/index.py`.
- Step 4: Custom Subdomain / Alias Assignment**
In Vercel Project Settings → General → Rename project to `ai-powered-threat-detection-system` to bind to target domain URL.

3. Option B: Physical Network Hardware Capture Deployment (On-Premise)

- Step 1: Network Tap / SPAN Port Mirroring**
Configure switch port mirroring (SPAN port) on core router to mirror incoming raw traffic to dedicated network interface card (e.g. `eth0` or `wlan0`).
- Step 2: Enable Live Socket Capture Module**
Run `traffic/live_capture.py` with administrative privileges (`root/sudo`):

```
sudo python3 -c "from traffic.live_capture import LiveNetworkCapture; from engine.correlation_engine import EnsembleCorrelationEngine; eng = EnsembleCorrelationEngine('models'); cap = LiveNetworkCapture(eng, interface='eth0'); cap.start_capture()"
```
- Step 3: Hardware Promiscuous Mode**
Ensure interface is in promiscuous mode (`sudo ip link set eth0 promisc on`) to capture all enterprise subnet packet frames.

4. Automated Active Mitigation Firewall Integration

The system generates executable shell mitigation commands tailored to each attack classification:

Attack Classification	Target Threat	Automated Active Firewall Command
DoS Attack	Single-Source Volumetric Flood	<code>sudo iptables -A INPUT -s {src_ip} -p tcp --dport 80 -m limit --limit 25/min -j ACCEPT && sudo iptables -A INPUT -s {src_ip} -j DROP</code>

DDoS Attack	Multi-Source Botnet Flood	sudo ip route add blackhole {src_ip}/32 && sudo iptables -I INPUT -s {src_ip}/24 -j DROP
PortScan	Reconnaissance Scanning	sudo iptables -A INPUT -s {src_ip} -m recent --name portscan --set -j DROP
BruteForce	SSH/FTP Credential Flood	sudo fail2ban-client set sshd banip {src_ip} && sudo iptables -A INPUT -s {src_ip} -p tcp --dport 22 -j DROP
ARP Spoofing / MITM	MAC Poisoning	sudo arptables -A INPUT --source-ip {src_ip} -j DROP && sudo ip neighbor flush all

5. ELK Stack (Elasticsearch, Logstash, Kibana) Integration

- Every detection payload is formatted into Elastic Common Schema (ECS 8.x) JSON by elk/elk_exporter.py.
- Configure Logstash pipeline by starting Logstash with elk/logstash.conf:

```
logstash -f elk/logstash.conf
```

- Events are automatically indexed into Elasticsearch under network-security-events-* for long-term SOC audit trails and Kibana visual analytics.